TIG Weld Assessment

Ritwij Parmar | Code (includes weights in the model folders)

I started with MobileNet because I'd used it in my deep learning course. I revisited the architecture before choosing V5. Its combination of convolution, attention and features at different scales seemed useful for weld texture and pool boundaries. Efficient inference was another reason I wanted to test it. It gave decent results, but performance varied across recordings.

I then tried DINOv3 as a benchmark to see whether its self-supervised features would transfer better. It gave fewer false alarms on assessment, but still missed defects, particularly lack of penetration. I could catch more by lowering the threshold, but that would also create more false alarms.

I had read about I-JEPA and understood the rough architecture, but this was my first time implementing it. Predicting hidden-region embeddings from the visible parts of an image seemed promising. I wanted to see whether adapting that process to weld images would give me better features.

One mistake I made was fully trusting the AI-generated code without checking it carefully. Its performance was decent, but weaker than I expected, so I went back and reinspected the implementation. That's when I found it was basically a normal classification network using I-JEPA's pretrained weights with LoRA and a classifier. The actual JEPA adaptation I wanted wasn't happening.

I corrected the training path so the context encoder and predictor learned through gradients, while the target encoder updated through EMA. After adaptation, I froze the resulting encoder and trained the classifier separately. The corrected pipeline gave me much stronger defect detection and the highest balanced accuracy in my matched benchmark.

Model results

Identical splits: 12 training recordings / 10,338 frames, 6 validation / 6,454 and 9 assessment / 6,763. Complete recordings stay in one split.

Model	Validation (%)			Assessment (%)		
	BA	Recall	FAR	BA	Recall	FAR
MobileNetV5-300M	92.55	88.53	3.43	79.57	81.51	22.38
DINOv3 ViT-L/16	89.70	85.91	6.51	88.00	83.45	7.45
I-JEPA ViT-H/14 - pretrained	94.78	89.72	0.16	84.89	85.02	15.25
I-JEPA ViT-H/14 - adapted	**98.41**	**97.58**	**0.77**	**92.75**	**96.51**	**11.00**

BA: balanced accuracy. Recall: defect recall. FAR: false-alarm rate. All are recording-weighted at threshold 0.5. Assessment was excluded from fitting but inspected in earlier research, so these are retrospective held-out results rather than a new blind test.

Later, I noticed that adapted I-JEPA's false-alarm rate rose from 0.77% on validation to 11.00% on assessment. When I checked the recordings, 514 of its 521 false alarms came from one recording, 170906-141809. The other eight contributed only seven false alarms. Across the good frames available in those recordings, the recording-weighted false-alarm rate was 0.48%. That recording also accounted for most of DINO's false alarms. I used SliceFinder to investigate the concentrated failure and found associations with bright-region position and contrast. That gave me a more specific appearance pattern to investigate, rather than a proven physical or label cause.

I followed this up with 15-epoch classifier trials using targeted augmentation, with and without extra weight on difficult good frames, while keeping the encoder frozen. I also screened a nonlinear classifier. None reduced false alarms while preserving the required defect recall in development, so I retained the original model.

GPU inference benchmark - selected adapted I-JEPA

Finally, I benchmarked the selected adapted I-JEPA on the server GPU. Compared with PyTorch FP32, TensorRT FP32 reduced model inference from 14.44 ms to 8.17 ms, about 1.77 times faster. Including image decoding and preprocessing, the speedup was 1.39 times, with identical decisions across all validation frames. Testing latency and memory use on an actual Jetson is future work, with latency as the main priority.

NVIDIA RTX PRO 6000 Blackwell Server Edition | Matched FP32, TF32 off | Batch 1 | 30 warmups, 200 timings | Median latency

Timing boundary	PyTorch FP32	TensorRT FP32	Speedup
GPU input to returned model logit	14.44 ms	8.17 ms	**1.77x**
PNG bytes to returned decision	22.24 ms	15.96 ms	**1.39x**

TensorRT matched all 6,454 validation decisions against the fresh PyTorch FP32 reference. PNG timing includes decoding and preprocessing but excludes camera acquisition, disk reads and startup. Historical model scores above remain unchanged.

Where I would focus in a real setup

If I were working with a real welding setup and had more control over data collection, I would prioritize more independent recordings under conditions similar to that difficult case, including both good and defective welds. Given how concentrated the false alarms were, my first suspicion would be a gap in how that case is represented in the data. I would test whether better coverage helps before spending more time changing the model.

My main focus would remain on the models. I would also look for ways to improve the whole pipeline like I did with TensorRT. In a ROS 2 setup I would test whether tuning Fast DDS or switching to Connext improves image delivery and reduces latency. I did similar work in my lab too and it helped a lot.